Name: Suyash Singh Tomar

BTech Cse, 3rd year

sec:"B"

CU24250178

ROll no. = 60

Experiment No. 6

Experiment Name

Predictive Analytics using Linear Regression and Model Performance Evaluation

Aim

To develop a Linear Regression model using Python for predicting continuous outcomes and evaluate its performance using appropriate regression metrics.

GITHUB LINK : https://github.com/Abhay34501/Mastery-in-Data-Analytics-and-Visualizations-and-Career-Advancement/blob/main/Experiment%206.py

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

sns.set_style("whitegrid")
plt.rcParams['figure.figsize'] = (8,5)
```

```
np.random.seed(42)

n = 200
TV = np.round(np.random.uniform(5, 300, n), 2)
Radio = np.round(np.random.uniform(0, 50, n), 2)
Newspaper = np.round(np.random.uniform(0, 100, n), 2)

# Sales depends mainly on TV & Radio with some noise
Sales = 4.5 + 0.045*TV + 0.19*Radio + 0.01*Newspaper + np.random.normal(0, 2, n)
```

```
Sales = np.round(Sales, 2)

df = pd.DataFrame({
    'TV': TV,
    'Radio': Radio,
    'Newspaper': Newspaper,
    'Sales': Sales
})

print(df.shape)
df.head()
```

```
(200, 4)
```

	TV	Radio	Newspaper	Sales
0	115.49	32.10	10.31	14.84
1	285.46	4.21	90.26	17.46
2	220.94	8.08	50.53	16.27
3	181.60	44.93	82.65	19.96
4	51.03	30.32	32.00	11.77

Next steps: Generate code with df | New interactive sheet

```
print(df.isnull().sum())
print(df.describe())

# No missing values, no categorical variables here.
# Selecting features and target
X = df[['TV', 'Radio', 'Newspaper']]
y = df['Sales']
```

```
TV           0
Radio        0
Newspaper    0
Sales        0
dtype: int64
                TV       Radio  Newspaper       Sales
count  200.000000  200.000000  200.00000  200.000000
mean   147.781650   25.218450   52.06730   16.343350
std     86.993071   14.649697   30.73093    5.034397
min      6.630000    0.250000    1.08000    2.120000
25%     72.435000   13.072500   25.54750   12.210000
50%    150.875000   27.080000   52.54000   16.730000
```

```
75%    228.275000   37.110000   81.11500   20.080000
max    296.130000   49.530000   99.97000   26.880000
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

print("Training set size:", X_train.shape)
print("Testing set size:", X_test.shape)
```

```
Training set size: (160, 3)
Testing set size: (40, 3)
```

```
model = LinearRegression()
model.fit(X_train, y_train)

print("Intercept:", model.intercept_)
print("Coefficients:", dict(zip(X.columns, model.coef_)))
```

```
Intercept: 4.330277344803875
Coefficients: {'TV': np.float64(0.04403000187706825), 'Radio': np.float64(0.1821091072856895), 'Newspaper': np.float64(0.017610127578564422)}
```

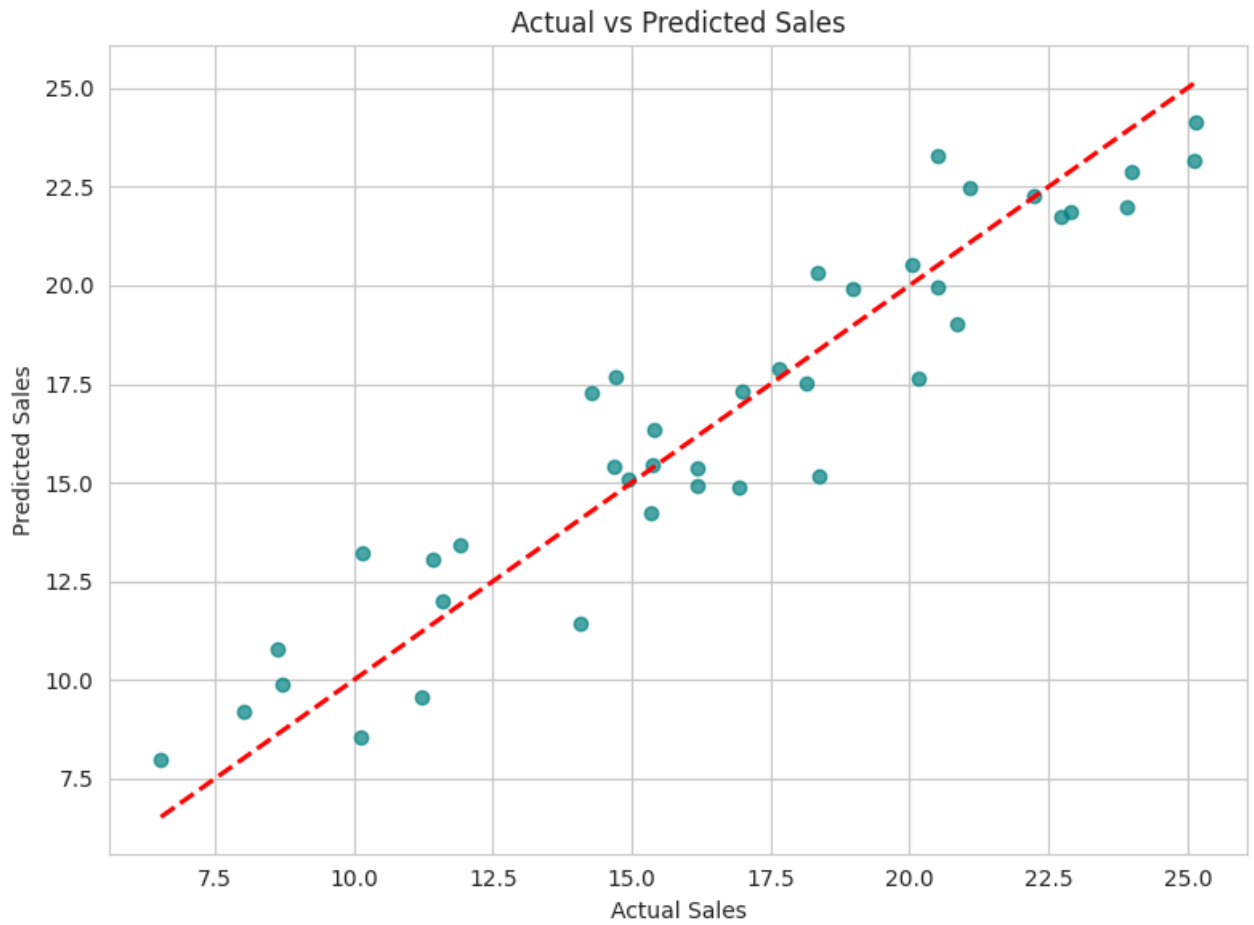
```
y_pred = model.predict(X_test)

results = pd.DataFrame({'Actual': y_test.values, 'Predicted': np.round(y_pred, 2)})
results.head(10)
```

	Actual	Predicted
0	15.39	16.34
1	8.71	9.88
2	18.97	19.93
3	18.13	17.53
4	10.13	8.56
5	22.23	22.25
6	22.72	21.74
7	18.38	15.17
8	14.94	15.10
9	20.52	23.28

Next steps: Generate code with `results` New interactive sheet

```
plt.figure(figsize=(8,6))
plt.scatter(y_test, y_pred, color='teal', alpha=0.7)
plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], 'r--', linewidth=2)
plt.xlabel('Actual Sales')
plt.ylabel('Predicted Sales')
plt.title('Actual vs Predicted Sales')
plt.tight_layout()
plt.show()
```

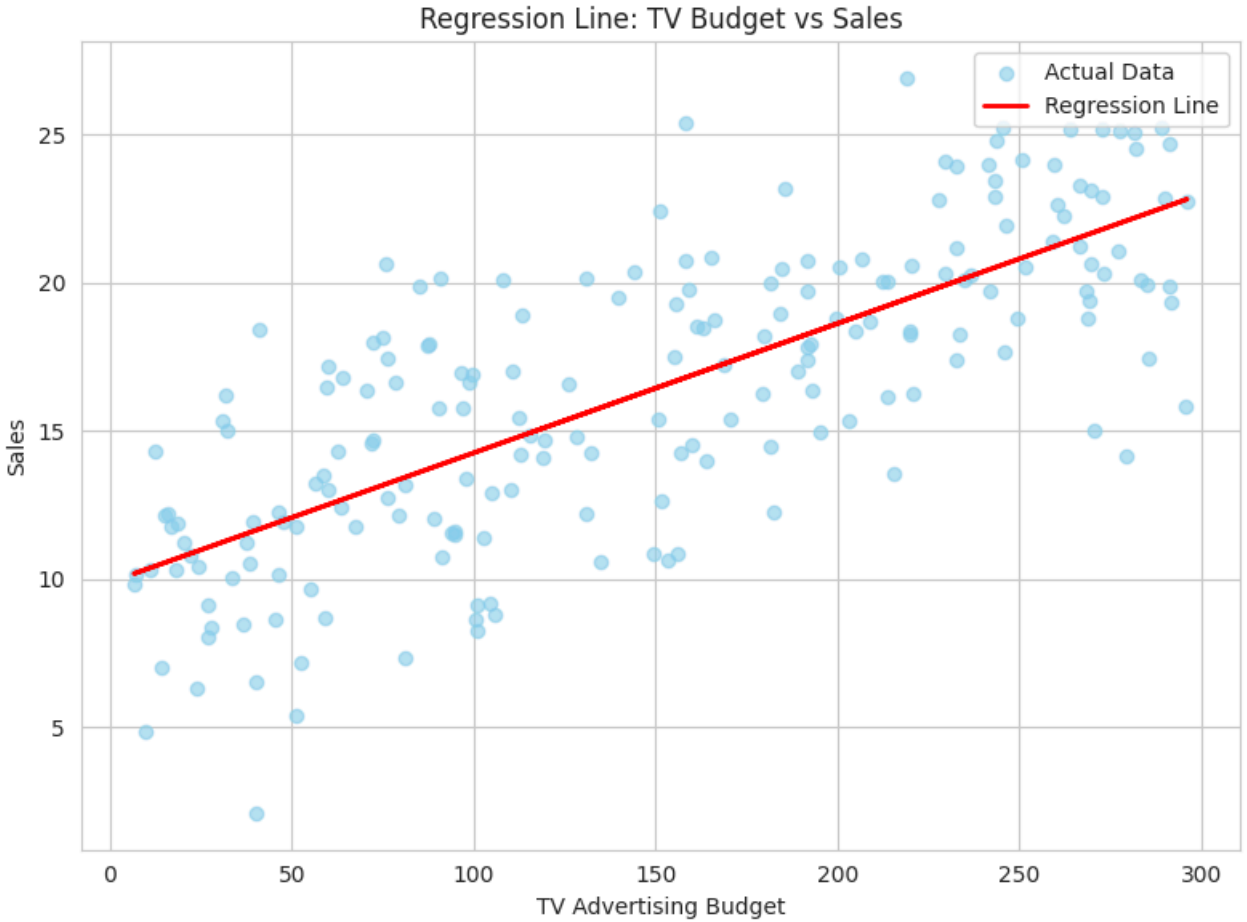



```
X_tv = df[['TV']]
y_tv = df['Sales']

model_tv = LinearRegression()
```

```
model_tv.fit(X_tv, y_tv)
y_tv_pred = model_tv.predict(X_tv)

plt.figure(figsize=(8,6))
plt.scatter(X_tv, y_tv, color='skyblue', alpha=0.6, label='Actual Data')
plt.plot(X_tv, y_tv_pred, color='red', linewidth=2, label='Regression Line')
plt.xlabel('TV Advertising Budget')
plt.ylabel('Sales')
plt.title('Regression Line: TV Budget vs Sales')
plt.legend()
plt.tight_layout()
plt.show()
```



```
mae = mean_absolute_error(y_test, y_pred)
mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
r2 = r2_score(y_test, y_pred)

print("MODEL PERFORMANCE METRICS")
print("="*40)
print(f"Mean Absolute Error (MAE)   : {mae:.3f}")
print(f"Mean Squared Error (MSE)    : {mse:.3f}")
print(f"Root Mean Squared Error    : {rmse:.3f}")
print(f"R² Score                    : {r2:.3f}")
```

```
MODEL PERFORMANCE METRICS
========================================
Mean Absolute Error (MAE)   : 1.410
Mean Squared Error (MSE)    : 2.763
Root Mean Squared Error    : 1.662
R² Score                    : 0.888
```

```
coef_df = pd.DataFrame({
    'Feature': X.columns,
    'Coefficient': model.coef_
}).sort_values(by='Coefficient', ascending=False)

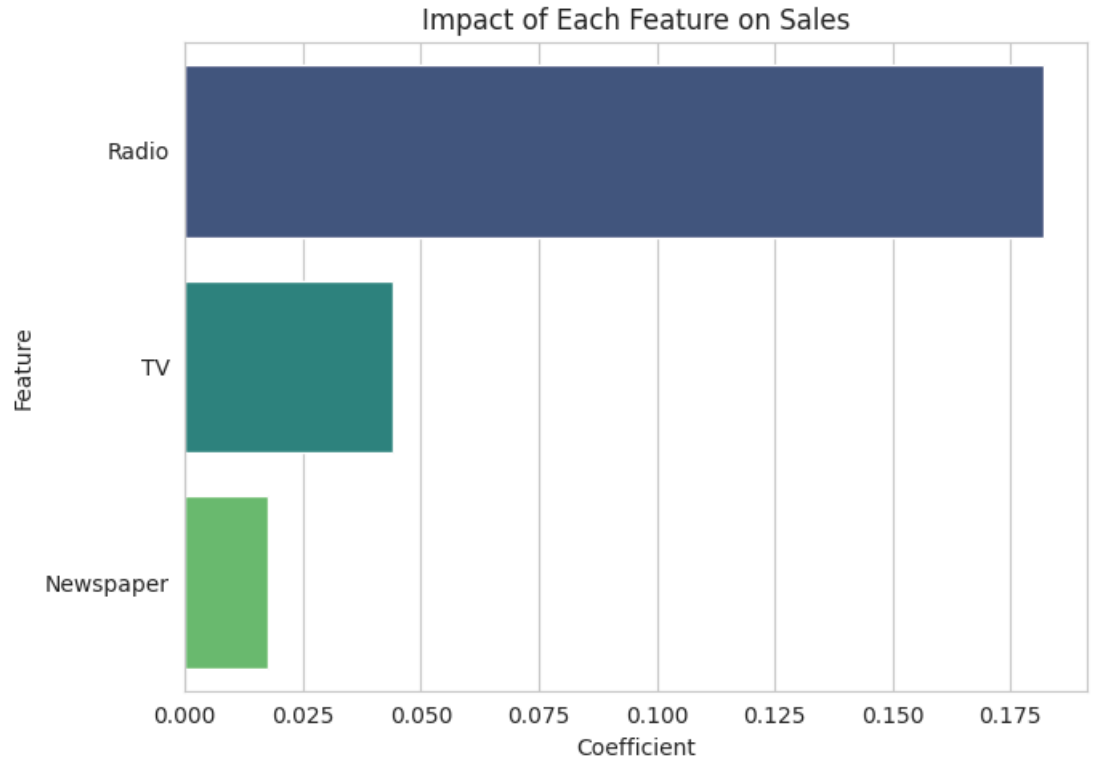
print(coef_df)

plt.figure(figsize=(7,5))
sns.barplot(x='Coefficient', y='Feature', data=coef_df, palette='viridis')
plt.title('Impact of Each Feature on Sales')
plt.tight_layout()
plt.show()
```

```
     Feature  Coefficient
1      Radio     0.182109
0         TV     0.044030
2  Newspaper     0.017610
/tmp/ipykernel_1592/3928973506.py:9: FutureWarning:

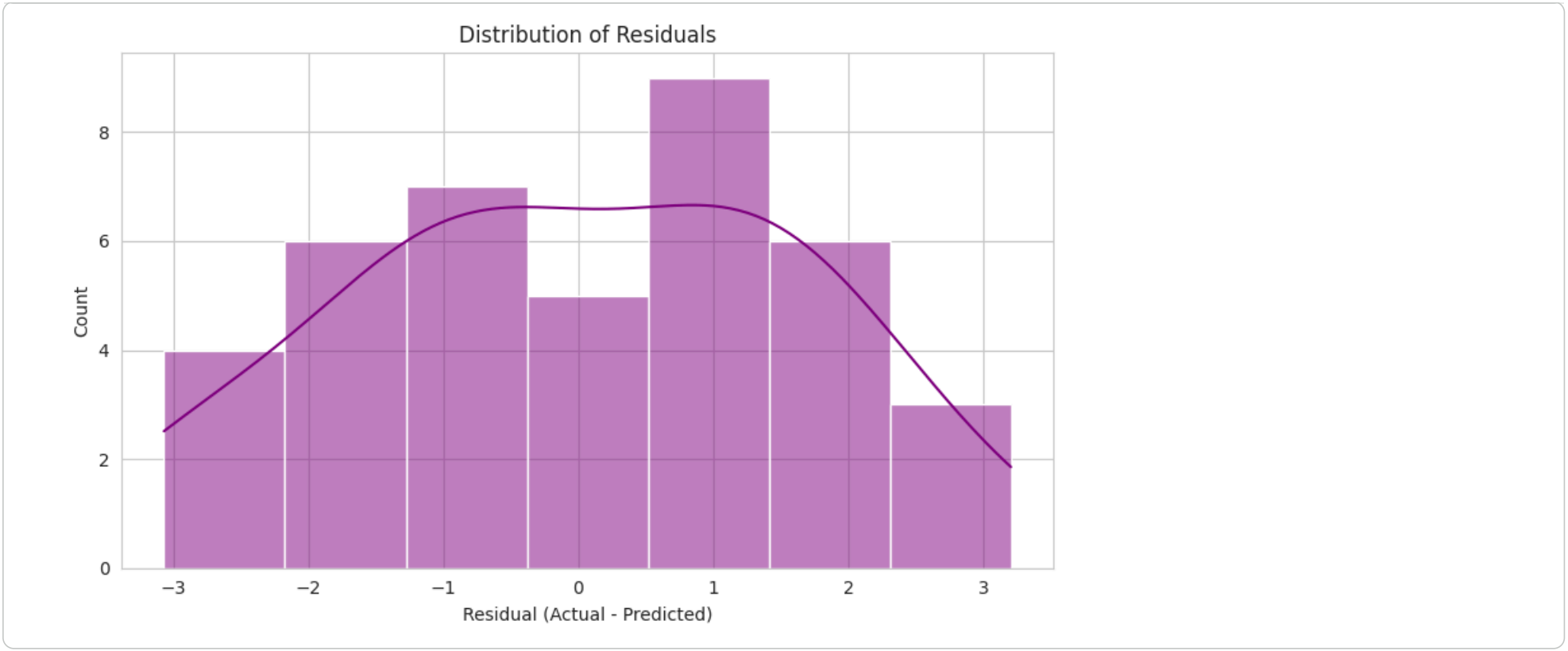
Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `y` variable to `hue` and set `legend=False` for the sa

  sns.barplot(x='Coefficient', y='Feature', data=coef_df, palette='viridis')
```




```
residuals = y_test - y_pred

plt.figure(figsize=(8,5))
sns.histplot(residuals, kde=True, color='purple')
plt.title('Distribution of Residuals')
plt.xlabel('Residual (Actual - Predicted)')
plt.tight_layout()
plt.show()
```




```
print("INTERPRETATION SUMMARY")
print("="*50)
print(f"R² Score of {r2:.2f} indicates that the model explains {r2*100:.1f}% "
      f"of the variance in Sales.")
print(f"RMSE of {rmse:.2f} shows the average prediction error in the same units as Sales.")
top_feature = coef_df.iloc[0]['Feature']
print(f"'{top_feature}' has the strongest positive impact on Sales based on coefficient magnitude.")
print("Linear Regression appears suitable here as the relationship between "
      "advertising spend and sales is approximately linear.")
```

```
INTERPRETATION SUMMARY
==================================================
R² Score of 0.89 indicates that the model explains 88.8% of the variance in Sales.
RMSE of 1.66 shows the average prediction error in the same units as Sales.
'Radio' has the strongest positive impact on Sales based on coefficient magnitude.
Linear Regression appears suitable here as the relationship between advertising spend and sales is approximately linear.
```

Question & Answer Section

Q1. What is Predictive Analytics, and how is it used in real-world applications?

Predictive analytics uses historical data, statistical algorithms, and machine learning techniques to forecast future outcomes or trends. It's widely used in business for sales forecasting, in finance for credit risk assessment and fraud detection, in healthcare for predicting disease risk, and in marketing for customer churn prediction — helping organizations make proactive, data-driven decisions.

Q2. Explain the working principle of the Linear Regression algorithm.

Linear Regression models the relationship between a dependent variable (target) and one or more independent variables (features) by fitting a straight line (or hyperplane in multiple dimensions) that minimizes the sum of squared differences between actual and predicted values. It estimates coefficients for each feature using the Ordinary Least Squares (OLS) method, producing an equation like y = b0 + b1*x1 + b2*x2 + ... + bn*xn that can then predict new outcomes.

Q3. Differentiate between dependent and independent variables with suitable examples.

The dependent variable (also called target or outcome) is the value being predicted — for example, Sales. Independent variables (also called features or predictors) are the inputs used to make that prediction — for example, TV advertising budget, Radio budget, and Newspaper budget. Changes in independent variables are assumed to influence the dependent variable, not the other way around.

Q4. Why is it necessary to split the dataset into training and testing sets?

Splitting the data ensures the model is evaluated on data it hasn't seen during training, giving an honest measure of how well it generalizes to new, unseen data. Without this split, a model could simply memorize the training data (overfitting) and appear highly accurate while actually performing poorly on real-world predictions.

Q5. What is the significance of the R^2 Score in regression analysis?

The R^2 Score (coefficient of determination) indicates the proportion of variance in the dependent variable that is explained by the independent variables, ranging from 0 to 1 (or negative for very poor models). An R^2 of 0.85, for instance, means 85% of the variability in the target variable is explained by the model — higher values indicate a better fit.

Q6. Differentiate between MAE, MSE, and RMSE. Which metric is more sensitive to large prediction errors?

MAE (Mean Absolute Error) calculates the average of absolute differences between actual and predicted values, treating all errors equally. MSE (Mean Squared Error) squares the errors before averaging, which penalizes larger errors more heavily. RMSE (Root Mean Squared Error) is the square root of MSE, bringing the error back to the same unit as the target variable while still being sensitive to large errors. MSE (and consequently RMSE) is more sensitive to large prediction errors because squaring amplifies bigger deviations disproportionately compared to MAE.

Q7. What assumptions should be satisfied before applying Linear Regression?

Key assumptions include: a linear relationship between independent and dependent variables, independence of errors (no autocorrelation), homoscedasticity (constant variance of residuals across all levels of the independent variables), normality of residuals, and no or minimal multicollinearity among independent variables.

Q8. How can overfitting and underfitting affect the performance of a regression model?

Overfitting occurs when a model learns the training data too precisely, including noise, resulting in excellent training performance but poor generalization to new data. Underfitting occurs when the model is too simple to capture the underlying pattern, leading to poor performance on both training and testing data. Both reduce the model's real-world predictive reliability — the goal is a balanced model that generalizes well.

Q9. Mention any three real-world applications of Linear Regression in business or industry.

Sales forecasting — predicting future sales based on advertising spend, seasonality, or economic indicators.

Real estate pricing — estimating house prices based on features like area, location, and number of rooms.

Risk assessment in insurance/finance — predicting claim amounts or credit risk scores based on customer attributes.

Q10. How can feature selection improve the accuracy and interpretability of a predictive model?

Feature selection removes irrelevant, redundant, or noisy variables from the model, which reduces overfitting, improves computational efficiency, and often increases predictive accuracy. It also makes the model more interpretable by focusing only on the features that genuinely impact the target variable, making it easier for stakeholders to understand which factors drive the predicted outcome.